LABORATORY PRACTICE III

PRACTICAL JOURNAL

Name	Prachi Borse
PRN No.	72311158J
Roll No.	A-20
Academic Year	2026–2027

Department of Computer Engineering

Genba Sopanrao Moze College of Engineering

Write a program to calculate Fibonacci numbers and find its step count.

```python
def fib_iterative(n):
    steps = 0
    if n == 0:
        steps += 1
        return 0, steps
    elif n == 1:
        steps += 1
        return 1, steps

    a, b = 0, 1
    for i in range(2, n + 1):
        steps += 1
        a, b = b, a + b
    return b, steps

n = int(input("Enter n: "))
result, steps = fib_iterative(n)
print(f"Fibonacci({n}) = {result}")
print(f"Step count (iterative) = {steps}")



# Recursive Fibonacci with step counter

steps = 0  # global step counter

def fib_recursive(n):
    global steps
    steps += 1  # count each call as a step
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib_recursive(n - 1) + fib_recursive(n - 2)

n = int(input("Enter n: "))
steps = 0
result = fib_recursive(n)
print(f"Fibonacci({n}) = {result}")

print(f"Step count (recursive) = {steps}")
```

```
PS D:\PRACTICAL\SEM-7\DAA> python -u "d:\PRACTICAL\SEM-7\DAA\Pr-1"
Enter n: 5
Fibonacci(5) = 5
Step count (iterative) = 4
Enter n: 5
Fibonacci(5) = 5
Step count (recursive) = 15
PS D:\PRACTICAL\SEM-7\DAA>
```

Implement Huffman Encoding using a greedy strategy OR implement job sequencing with deadlines using a greedy method.

```python
class Job:
    def __init__(self, job_id, deadline, profit):
        self.job_id = job_id
        self.deadline = deadline
        self.profit = profit

def job_sequencing(jobs):
    # Step 1: Sort jobs by decreasing profit
    jobs.sort(key=lambda x: x.profit, reverse=True)

    # Step 2: Find maximum deadline
    max_deadline = max(job.deadline for job in jobs)

    # Step 3: Initialize time slots (None = free)
    slots = [None] * (max_deadline + 1)
    total_profit = 0

    # Step 4: Assign jobs to slots
    for job in jobs:
        for slot in range(job.deadline, 0, -1):
            if slots[slot] is None:
                slots[slot] = job.job_id
                total_profit += job.profit
                break

    # Display result
    scheduled_jobs = [job for job in slots if job is not None]
    print("Scheduled Jobs:", scheduled_jobs)
    print("Total Profit:", total_profit)


# --- Driver Code ---
if __name__ == "__main__":
    jobs = [
        Job('J1', 2, 100),
        Job('J2', 1, 19),
        Job('J3', 2, 27),
        Job('J4', 1, 25),
        Job('J5', 3, 15)
    ]

    job_sequencing(jobs)
```

```
PS D:\PRACTICAL\SEM-7\DAA> python -u "d:\PRACTICAL\SEM-7\DAA\Pr-2"
Scheduled Jobs: ['J3', 'J1', 'J5']
Total Profit: 142
PS D:\PRACTICAL\SEM-7\DAA>
```

Write a program to solve a Fractional Knapsack problem using a greedy method.

```python
class Item:
    def __init__(self, value, weight):   # ✅ Correct constructor
        self.value = value
        self.weight = weight
        self.ratio = value / weight  # value-to-weight ratio


def fractional_knapsack(capacity, values, weights):
    # Step 1: Create list of items
    items = [Item(values[i], weights[i]) for i in range(len(values))]

    # Step 2: Sort items by value/weight ratio in descending order
    items.sort(key=lambda x: x.ratio, reverse=True)

    total_value = 0.0  # Final profit

    # Step 3: Take items until knapsack is full
    for item in items:
        if capacity >= item.weight:
            # Take full item
            capacity -= item.weight
            total_value += item.value
        else:
            # Take fraction of item
            total_value += item.value * (capacity / item.weight)
            break  # Knapsack is full

    return total_value


# Driver code
if __name__ == "__main__":   # ✅ Correct main check
    values = [60, 100, 120]   # Profits
    weights = [10, 20, 30]    # Weights
    capacity = 50             # Knapsack

    max_value = fractional_knapsack(capacity, values, weights)
    print("Maximum value in Knapsack =", max_value)  # ✅ Fixed print
```

```
PS D:\PRACTICAL\SEM-7\DAA> python -u "d:\PRACTICAL\SEM-7\DAA\Pr-3"
Maximum value in Knapsack = 240.0
PS D:\PRACTICAL\SEM-7\DAA>
```

Write a program to solve a 0-1 Knapsack problem using dynamic programming or branch and bound strategy.

```python
def knapsack(weights, values, capacity):
    n = len(values)

    # Step 1: Create DP table
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    # Step 2: Fill DP table
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w - weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]

    # Step 3: Backtrack to find selected items
    selected_items = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i-1][w]:
            selected_items.append(i)  # item i was included
            w -= weights[i-1]

    selected_items.reverse()  # To show items in original order

    # Step 4: Return results
    return dp[n][capacity], selected_items


# Example usage
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
capacity = 5

max_value, items = knapsack(weights, values, capacity)
print("Maximum value in Knapsack =", max_value)
print("Items included (1-indexed) =", items)
```

```
PS D:\PRACTICAL\SEM-7\DAA> python -u "d:\PRACTICAL\SEM-7\DAA\Pr-4"
Maximum value in Knapsack = 7
Items included (1-indexed) = [1, 2]
PS D:\PRACTICAL\SEM-7\DAA>
```

Design an 8-Queens matrix with the first Queen placed; use backtracking to place the remaining Queens.

```python
N = 8
def print_board(board):
    for row in board:
        print(" ".join("Q" if x else "." for x in row))
    print("\n")

def is_safe(board, row, col):
    # Check column
    for i in range(row):
        if board[i][col]:
            return False

    # Check upper left diagonal
    for i, j in zip(range(row-1, -1, -1), range(col-1, -1, -1)):
        if board[i][j]:
            return False

    # Check upper right diagonal
    for i, j in zip(range(row-1, -1, -1), range(col+1, N)):
        if board[i][j]:
            return False

    return True

def solve(board, row=1):
    # Base case: all queens placed
    if row == N:
        print("Final 8-Queens Solution:\n")
        print_board(board)
        return True

    # Try placing queen in each column for the current row
    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1
            if solve(board, row + 1):
                return True
            board[row][col] = 0  # Backtrack

    return False

# Initialize board with all 0s
board = [[0] * N for _ in range(N)]

# Pre-place first Queen
board[0][0] = 1

print("Starting with first Queen at (0, 0):\n")
print_board(board)

# Solve for remaining Queens
if not solve(board):
    print("No solution found!")
```

Design an 8-Queens matrix with the first Queen placed; use backtracking to place the remaining Queens.

```
PS D:\PRACTICAL\SEM-7\DAA> python -u "d:\PRACTICAL\SEM-7\DAA\Pr-6"
Starting with first Queen at (0, 0):

Q . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .


Final 8-Queens Solution:

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

Predict the price of an Uber ride using pickup and drop-off details; preprocess data, identify outliers, check correlation, and compare Linear Regression and Random Forest Regression.

In [28]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

df = pd.read_csv("C:/Users/prati/OneDrive/문서/uber.csv")
df.head()
print(df.info())
print("-------------------------------------")
#count of missing values
print(df.isnull().sum())
df = df.dropna()
```

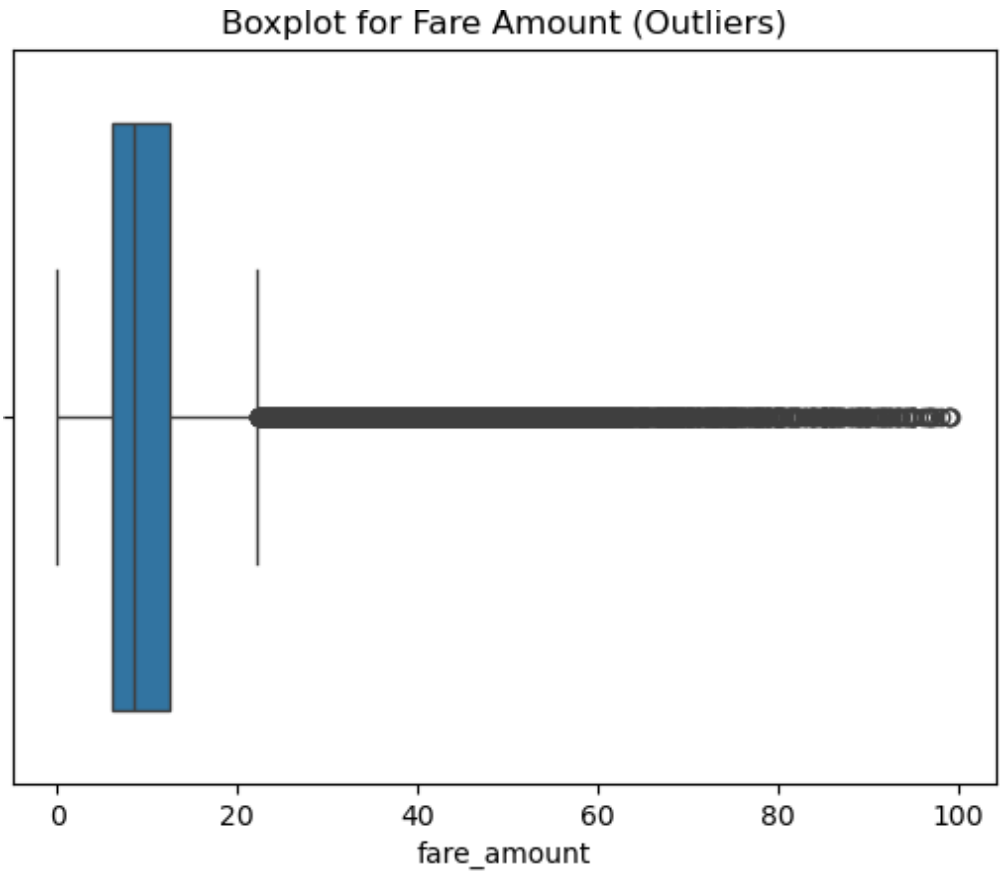
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 9 columns):
 #   Column             Non-Null Count   Dtype  
---  ------             --------------   -----  
 0   Unnamed: 0         200000 non-null  int64  
 1   key                200000 non-null  object 
 2   fare_amount        200000 non-null  float64
 3   pickup_datetime    200000 non-null  object 
 4   pickup_longitude   200000 non-null  float64
 5   pickup_latitude    200000 non-null  float64
 6   dropoff_longitude  199999 non-null  float64
 7   dropoff_latitude   199999 non-null  float64
 8   passenger_count    200000 non-null  int64  
dtypes: float64(5), int64(2), object(2)
memory usage: 13.7+ MB
None
-------------------------
Unnamed: 0           0
key                  0
fare_amount          0
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    1
dropoff_latitude     1
passenger_count      0
dtype: int64
```

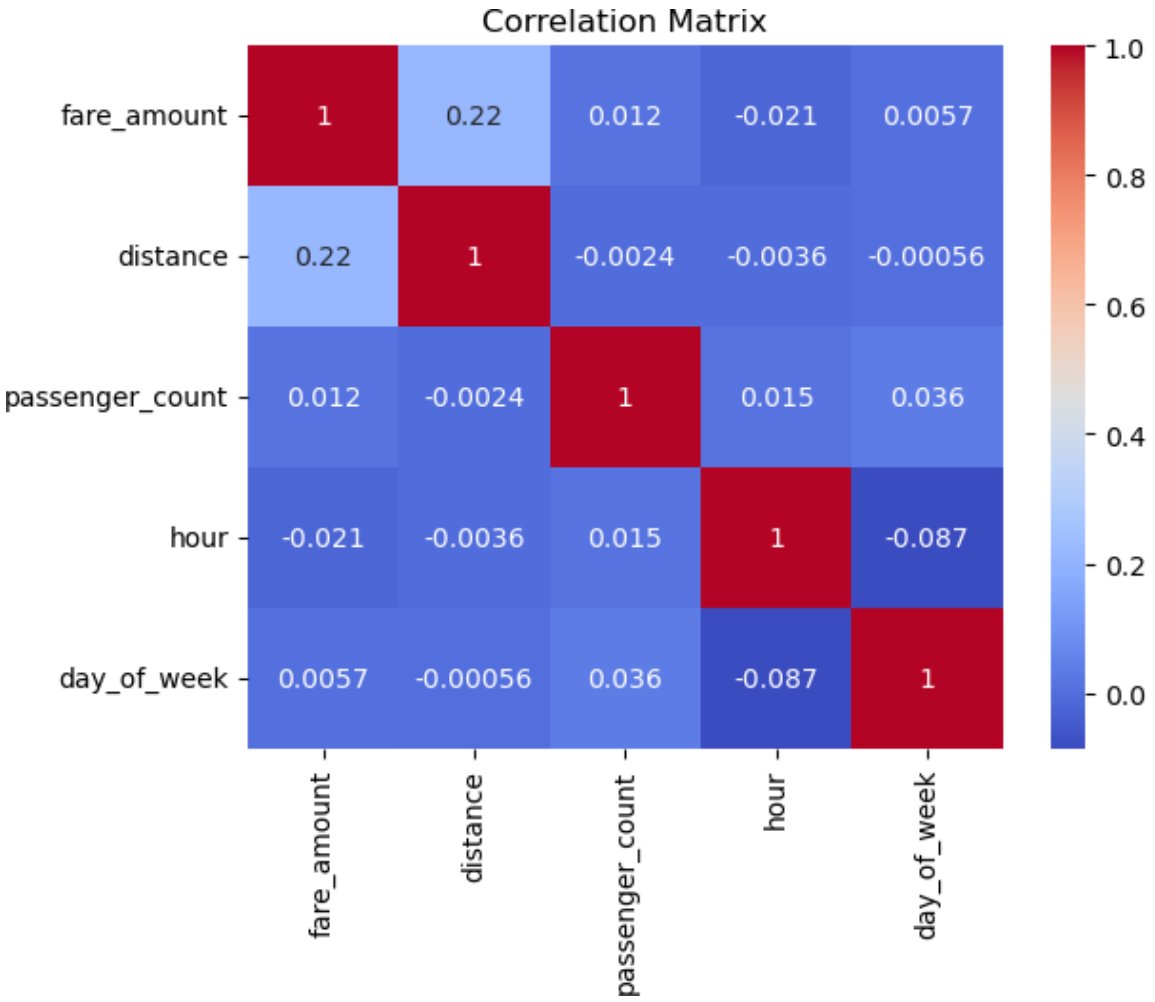
In [29]:
```
# Calculate distance using simple Euclidean formula
df["distance"] = (
    ((df['dropoff_latitude'] - df['pickup_latitude'])**2 +
     (df['dropoff_longitude'] - df['pickup_longitude'])**2) * 0.5
```

In [30]:
```
# Extract time features
df['pickup_datetime'] = pd.to_datetime(df['pickup_datetime'])
df['hour'] = df['pickup_datetime'].dt.hour
df['day_of_week'] = df['pickup_datetime'].dt.dayofweek
```

In [31]:
```
# Boxplot for fare amount
sns.boxplot(x=df['fare_amount'])
plt.title("Boxplot for Fare Amount (Outliers)")
plt.show()

# Correlation heatmap
corr = df[['fare_amount', 'distance', 'passenger_count', 'hour', 'day_of_week']]
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```





In [32]:
```python
# Feature and target selection
X = df[['distance', 'passenger_count', 'hour', 'day_of_week']]
y = df['fare_amount']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
```

In [33]:
```python
# Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

# Evaluation function
def evaluate(y_true, y_pred, model_name):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    print(f"{model_name} Results:")
    print(f"  R² Score: {r2:.4f}")
    print(f"   RMSE: {rmse:.4f}\n")
```

In [34]:
```
evaluate(y_test, y_pred_lr, "Linear Regression")
evaluate(y_test, y_pred_rf, "Random Forest Regression")
```

```
Linear Regression Results:
  R² Score: 0.0663
  RMSE: 9.0284

Random Forest Regression Results:
  R² Score: 0.7545
  RMSE: 4.6290
```

In [ ]:
```

```

Implement K-Means clustering or Hierarchical Clustering on sales_data_sample.csv and determine the number of clusters using the Elbow Method.

In [30]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [31]:
```
df = pd.read_csv('sales_data_sample.csv',encoding='unicode_escape')
df.head()
```

Out[31]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	OR
0	10107	30	95.70	2	2871.00	
1	10121	34	81.35	5	2765.90	
2	10134	41	94.74	2	3884.34	
3	10145	45	83.26	6	3746.70	
4	10159	49	100.00	14	5205.27	1

5 rows × 25 columns

In [32]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   ORDERNUMBER       2823 non-null   int64  
 1   QUANTITYORDERED   2823 non-null   int64  
 2   PRICEEACH         2823 non-null   float64
 3   ORDERLINENUMBER   2823 non-null   int64  
 4   SALES             2823 non-null   float64
 5   ORDERDATE         2823 non-null   object 
 6   STATUS            2823 non-null   object 
 7   QTR_ID            2823 non-null   int64  
 8   MONTH_ID          2823 non-null   int64  
 9   YEAR_ID           2823 non-null   int64  
 10  PRODUCTLINE       2823 non-null   object 
 11  MSRP              2823 non-null   int64  
 12  PRODUCTCODE       2823 non-null   object 
 13  CUSTOMERNAME      2823 non-null   object 
 14  PHONE             2823 non-null   object 
 15  ADDRESSLINE1      2823 non-null   object 
 16  ADDRESSLINE2      302 non-null    object 
 17  CITY              2823 non-null   object 
 18  STATE             1337 non-null   object 
 19  POSTALCODE        2747 non-null   object 
 20  COUNTRY           2823 non-null   object 
 21  TERRITORY         1749 non-null   object 
 22  CONTACTLASTNAME   2823 non-null   object 
 23  CONTACTFIRSTNAME  2823 non-null   object 
 24  DEALSIZE          2823 non-null   object 
dtypes: float64(2), int64(7), object(16)
memory usage: 551.5+ KB
```

In [33]:
```
df_drop  = ['ADDRESSLINE1', 'ADDRESSLINE2', 'POSTALCODE', 'CITY', 'TERRITORY', '
df = df.drop(df_drop, axis=1)
```

In [34]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 14 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   QUANTITYORDERED   2823 non-null   int64  
 1   PRICEEACH         2823 non-null   float64
 2   ORDERLINENUMBER   2823 non-null   int64  
 3   SALES             2823 non-null   float64
 4   ORDERDATE         2823 non-null   object 
 5   STATUS            2823 non-null   object 
 6   QTR_ID            2823 non-null   int64  
 7   MONTH_ID          2823 non-null   int64  
 8   YEAR_ID           2823 non-null   int64  
 9   PRODUCTLINE       2823 non-null   object 
 10  MSRP              2823 non-null   int64  
 11  PRODUCTCODE       2823 non-null   object 
 12  COUNTRY           2823 non-null   object 
 13  DEALSIZE          2823 non-null   object 
dtypes: float64(2), int64(6), object(6)
memory usage: 308.9+ KB
```

In [35]:
```
for col in df.columns.values:
    print(df[col].value_counts())
```

```
34    112
21    103
46    101
27    100
31     97
41     97
45     97
26     96
29     94
48     94
25     94
20     93
33     92
22     92
32     91
24     91
38     91
49     91
36     89
44     89
37     87
43     85
39     84
28     82
40     78
42     76
30     75
23     73
35     71
47     70
50     65
55     16
66      5
15      4
51      4
61      3
18      3
60      3
76      3
59      3
56      3
19      3
64      3
10      2
6       2
11      2
54      2
70      2
97      1
85      1
62      1
52      1
16      1
13      1
58      1
65      1
12      1
77      1
Name: QUANTITYORDERED,  dtype: int64
100.00    1304
```

```
59.87        6
96.34        6
57.73        5
80.55        5
          ...
48.30        1
87.96        1
36.21        1
98.48        1
62.24        1
Name: PRICEEACH, Length: 1016, dtype:  int64
1     307
2     291
3     270
4     256
5     239
6     221
7     197
8     187
9     165
10    141
11    128
12    110
13     97
14     81
15     56
16     42
17     25
18     10
Name: ORDERLINENUMBER, dtype: int64
3003.00    3
5464.69    2
2257.92    2
5004.80    2
2172.48    2
          ..
2312.24    1
2793.71    1
1908.28    1
3441.37    1
2116.16    1
Name: SALES, Length: 2763, dtype: int64
11/14/2003  0:00    38
11/24/2004  0:00    35
11/12/2003  0:00    34
11/17/2004  0:00    32
11/4/2004 0:00      29
                    ..
4/20/2004 0:00       1
8/4/2004 0:00        1
2/2/2004 0:00        1
8/28/2004 0:00       1
4/21/2003 0:00       1
Name: ORDERDATE, Length: 252, dtype: int64
Shipped        2617
Cancelled        60
Resolved         47
On Hold          44
In Process       41
Disputed         14
```

```
Name: STATUS, dtype: int64
4    1094
1     665
2     561
3     503
Name: QTR_ID, dtype: int64
11    597
10    317
5     252
1     229
2     224
3     212
8     191
12    180
4     178
9     171
7     141
6     131
Name: MONTH_ID, dtype: int64
2004    1345
2003    1000
2005     478
Name: YEAR_ID, dtype: int64
Classic Cars         967
Vintage Cars         607
Motorcycles          331
Planes               306
Trucks and Buses     301
Ships                234
Trains                77
Name: PRODUCTLINE,  dtype: int64
118    104
99     103
136     80
62      78
68      77
      ...
73      23
41      22
170     22
71      22
92      22
Name: MSRP, Length: 80, dtype: int64
S18_3232    52
S10_1949    28
S24_1444    28
S10_4962    28
S24_2840    28
            ..
S18_1749    22
S24_2887    22
S24_3969    22
S18_4409    22
S18_4933    22
Name: PRODUCTCODE, Length: 109, dtype: int64
USA             1004
Spain            342
France           314
Australia        185
UK               144
```

```
Italy            113
Finland           92
Norway            85
Singapore         79
Canada            70
Denmark           63
Germany           62
Sweden            57
Austria           55
Japan             52
Belgium           33
Switzerland       31
Philippines       26
Ireland           16
Name: COUNTRY, dtype: int64
Medium    1384
Small     1282
Large      157
Name: DEALSIZE, dtype: int64
```

In [36]:
```
df.drop(columns=['ORDERDATE','STATUS','MONTH_ID','QTR_ID','YEAR_ID'],inplace=Tru
df.head()
```

Out[36]:

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	PRODUCTLINE	MSR
0	30	95.70	2	2871.00	Motorcycles	9
1	34	81.35	5	2765.90	Motorcycles	9
2	41	94.74	2	3884.34	Motorcycles	9
3	45	83.26	6	3746.70	Motorcycles	9
4	49	100.00	14	5205.27	Motorcycles	9

In [37]:
```
from sklearn.preprocessing import LabelEncoder
def convert_categories(col):
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].values)
```

In [38]:
```
categories = ['PRODUCTLINE','PRODUCTCODE','COUNTRY','DEALSIZE']
for col in categories:
    convert_categories(col)
```

In [41]:
```
df.head()
```

Out[41]:

	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	PRODUCTLINE	MSR
0	30	95.70	2	2871.00	1	9
1	34	81.35	5	2765.90	1	9
2	41	94.74	2	3884.34	1	9
3	45	83.26	6	3746.70	1	9
4	49	100.00	14	5205.27	1	9

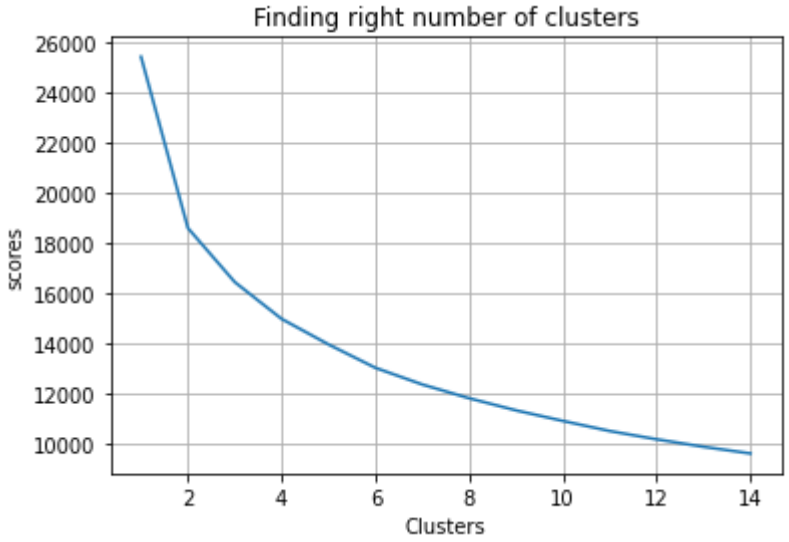
In [42]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
data = sc.fit_transform(df)
```

Elbow Method

Finding optimal numbers of clusters is elbow method For each value of K, we are calculating WCSS (Within-Cluster Sum of Square). WCSS is the sum of squared distance between each point and the centroid in a cluster.When we plot the WCSS with the K value, the plot looks like an Elbow

In [44]:
```
from sklearn.cluster import KMeans
wcss = []
for k in range(1,15):
    kmeans = KMeans(n_clusters=k,init='k-means++',random_state=15)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)
```

In [49]:
```
k = list(range(1,15))
plt.plot(k,wcss)
plt.xlabel('Clusters')
plt.ylabel('scores')
plt.title('Finding right number of clusters')
plt.grid()
plt.show()
```



At k=4, the graph starts to move almost parallel to the X-axis. The K value corresponding to this point is the optimal K value or an optimal number of clusters.

In [ ]:
```
```

Implement K-Nearest Neighbors (KNN) on diabetes.csv and calculate the confusion matrix, accuracy, error rate, precision, and recall.

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, r
```

In [3]:
```
df = pd.read_csv("C:/Users/LOQ/Downloads/3rd)K-Nearest Neighbors (KNN)/diabetes.
print(df.head())
```

```
   Pregnancies   Glucose   BloodPressure   SkinThickness   Insulin    BMI   \
0             6       148              72              35         0  33.6
1             1        85              66              29         0  26.6
2             8       183              64               0         0  23.3
3             1        89              66              23        94  28.1
4             0       137              40              35       168  43.1

   DiabetesPedigreeFunction   Age   Outcome
0                     0.627    50         1
1                     0.351    31         0
2                     0.672    32         1
3                     0.167    21         0
4                     2.288    33         1
```

In [4]:
```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

In [5]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

In [6]:
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [7]:
```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

Out[7]:

KNeighborsClassifier

▼ Parameters

n_neighbors		5
weights		'uniform'
algorithm		'auto'
leaf_size		30
p		2

In [8]:
```
y_pred = knn.predict(X_test)
```

In [9]:
```
cm = confusion_matrix(y_test, y_pred)
```

In [10]:
```
accuracy = accuracy_score(y_test, y_pred)
```

In [11]:
```
error_rate = 1 - accuracy
```

In [12]:
```
precision = precision_score(y_test, y_pred)
```

In [13]:
```
recall = recall_score(y_test, y_pred)
```

In [14]:
```
print("Confusion Matrix:\n", cm)
print("Accuracy:", round(accuracy, 4))
print("Error Rate:", round(error_rate, 4))
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
```

```
Confusion Matrix:
 [[79 20]
 [27 28]]
Accuracy: 0.6948
Error Rate: 0.3052
Precision: 0.5833
Recall: 0.5091
```

In [ ]:

Build a neural-network classifier to predict whether a bank customer will leave within the next six months; evaluate its performance.

In [ ]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import tensorflow as tf
```

In [4]:
```
df = pd.read_csv('Churn_Modelling.csv')
df.head()
```

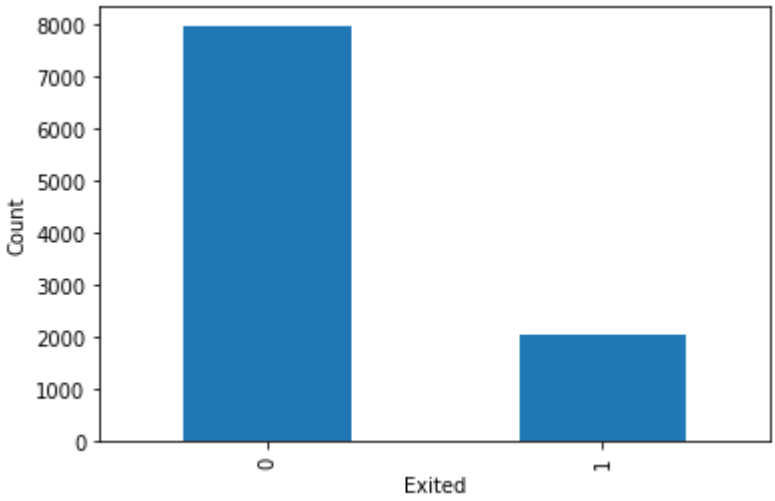
Out[4]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure
0	1	15634602	Hargrave	619	France	Female	42	2
1	2	15647311	Hill	608	Spain	Female	41	1
2	3	15619304	Onio	502	France	Female	42	8
3	4	15701354	Boni	699	France	Female	39	1
4	5	15737888	Mitchell	850	Spain	Female	43	2

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   RowNumber        10000 non-null  int64  
 1   CustomerId       10000 non-null  int64  
 2   Surname          10000 non-null  object 
 3   CreditScore      10000 non-null  int64  
 4   Geography        10000 non-null  object 
 5   Gender           10000 non-null  object 
 6   Age              10000 non-null  int64  
 7   Tenure           10000 non-null  int64  
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64  
 10  HasCrCard        10000 non-null  int64  
 11  IsActiveMember   10000 non-null  int64  
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64  
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

In [6]:
```
plt.xlabel('Exited')
plt.ylabel('Count')
df['Exited'].value_counts().plot.bar()
plt.show()
```



In [7]:
```
df['Geography'].value_counts()
```

Out[7]:
```
France     5014
Germany    2509
Spain      2477
Name: Geography, dtype: int64
```

In [8]:
```
df  =  pd.concat([df,pd.get_dummies(df['Geography'],prefix='Geo')],axis=1)
```

In [9]:
```
df  =  pd.concat([df,pd.get_dummies(df['Gender'])],axis=1)
```

In [10]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 19 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   RowNumber        10000 non-null  int64  
 1   CustomerId       10000 non-null  int64  
 2   Surname          10000 non-null  object 
 3   CreditScore      10000 non-null  int64  
 4   Geography        10000 non-null  object 
 5   Gender           10000 non-null  object 
 6   Age              10000 non-null  int64  
 7   Tenure           10000 non-null  int64  
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64  
 10  HasCrCard        10000 non-null  int64  
 11  IsActiveMember   10000 non-null  int64  
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64  
 14  Geo_France       10000 non-null  uint8  
 15  Geo_Germany      10000 non-null  uint8  
 16  Geo_Spain        10000 non-null  uint8  
 17  Female           10000 non-null  uint8  
 18  Male             10000 non-null  uint8  
dtypes: float64(2), int64(9), object(3), uint8(5)
memory usage: 1.1+ MB
```

In [11]:
```
df.drop(columns=['RowNumber','CustomerId','Surname','Geography','Gender'],inplac
```

In [12]:
```
df.head()
```

Out[12]:

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember
0	619	42	2	0.00	1	1	1
1	608	41	1	83807.86	1	0	1
2	502	42	8	159660.80	3	1	0
3	699	39	1	0.00	2	0	0
4	850	43	2	125510.82	1	1	1

Splitting Data

In [13]:
```
y = df['Exited'].values
x = df.loc[:,df.columns != 'Exited'].values
```

In [14]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,random_state=20,test_size=0
```

Scaling Data

In [15]:
```
from sklearn.preprocessing import StandardScaler
std_x = StandardScaler()
x_train = std_x.fit_transform(x_train)
x_test = std_x.transform(x_test)
```

In [16]:
```
x_train.shape
```

Out[16]:
```
(7500, 13)
```

Tensorflow Model - Neural Network Classifier

In [17]:
```
import tensorflow as tf
from tensorflow.keras.layers import Dense,Conv1D,Flatten
from tensorflow.keras.models import Sequential, Model
```

In [18]:
```
model=Sequential()
model.add(Flatten(input_shape=(13,)))
model.add(Dense(100,activation='relu'))
model.add(Dense(1,activation='sigmoid'))
```

In [19]:
```
model.compile(optimizer='adam',metrics=['accuracy'],loss='BinaryCrossentropy')
```

In [20]:
```
model.fit(x_train,y_train,batch_size=64,validation_split=0.1,epochs=100)
```

```
Epoch 1/100
106/106 [==============================] - 2s 2ms/step - loss: 0.4951 - accuracy:
0.7816 - val_loss: 0.4189 - val_accuracy: 0.8267
Epoch 2/100
106/106 [==============================] - 0s 1ms/step - loss: 0.4271 - accuracy:
0.8121 - val_loss: 0.3973 - val_accuracy: 0.8413
Epoch 3/100
106/106 [==============================] - 0s 1ms/step - loss: 0.4093 - accuracy:
0.8239 - val_loss: 0.3797 - val_accuracy: 0.8400
Epoch 4/100
106/106 [==============================] - 0s 982us/step - loss: 0.3929 - accurac
y: 0.8326 - val_loss: 0.3654 - val_accuracy: 0.8560
Epoch 5/100
106/106 [==============================] - 0s 952us/step - loss: 0.3792 - accurac
y: 0.8397 - val_loss: 0.3482 - val_accuracy: 0.8627
Epoch 6/100
106/106 [==============================] - 0s 993us/step - loss: 0.3683 - accurac
y: 0.8431 - val_loss: 0.3421 - val_accuracy: 0.8787
Epoch 7/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3620 - accuracy:
0.8479 - val_loss: 0.3311 - val_accuracy: 0.8720
Epoch 8/100
106/106 [==============================] - 0s 975us/step - loss: 0.3564 - accurac
y: 0.8508 - val_loss: 0.3316 - val_accuracy: 0.8747
Epoch 9/100
106/106 [==============================] - 0s 975us/step - loss: 0.3534 - accurac
y: 0.8532 - val_loss: 0.3232 - val_accuracy: 0.8733
Epoch 10/100
106/106 [==============================] - 0s 933us/step - loss: 0.3507 - accurac
y: 0.8553 - val_loss: 0.3258 - val_accuracy: 0.8787
Epoch 11/100
106/106 [==============================] - 0s 977us/step - loss: 0.3485 - accurac
y: 0.8557 - val_loss: 0.3198 - val_accuracy: 0.8787
Epoch 12/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3466 - accuracy:
0.8591 - val_loss: 0.3172 - val_accuracy: 0.8720
Epoch 13/100
106/106 [==============================] - 0s 994us/step - loss: 0.3452 - accurac
y: 0.8570 - val_loss: 0.3170 - val_accuracy: 0.8827
Epoch 14/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3448 - accuracy:
0.8561 - val_loss: 0.3203 - val_accuracy: 0.8773
Epoch 15/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3425 - accuracy:
0.8597 - val_loss: 0.3159 - val_accuracy: 0.8787
Epoch 16/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3411 - accuracy:
0.8578 - val_loss: 0.3169 - val_accuracy: 0.8773
Epoch 17/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3411 - accuracy:
0.8585 - val_loss: 0.3155 - val_accuracy: 0.8747
Epoch 18/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3398 - accuracy:
0.8579 - val_loss: 0.3195 - val_accuracy: 0.8747
Epoch 19/100
106/106 [==============================] - 0s 982us/step - loss: 0.3386 - accurac
y: 0.8604 - val_loss: 0.3146 - val_accuracy: 0.8827
Epoch 20/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3377 - accuracy:
0.8601 - val_loss: 0.3171 - val_accuracy: 0.8827
```

```
Epoch 21/100
106/106 [==============================] - 0s 983us/step - loss: 0.3377 - accurac
y: 0.8610 - val_loss: 0.3184 - val_accuracy: 0.8747
Epoch 22/100
106/106 [==============================] - 0s 1ms/step  - loss: 0.3372 - accuracy:
0.8597 - val_loss: 0.3155 - val_accuracy: 0.8800
Epoch 23/100
106/106 [==============================] - 0s 1ms/step  - loss: 0.3363 - accuracy:
0.8610 - val_loss: 0.3185 - val_accuracy: 0.8760
Epoch 24/100
106/106 [==============================] - 0s 1ms/step  - loss: 0.3346 - accuracy:
0.8604 - val_loss: 0.3147 - val_accuracy: 0.8800
Epoch 25/100
106/106 [==============================] - 0s 979us/step  - loss: 0.3348  - accurac
y: 0.8599 - val_loss: 0.3160 - val_accuracy: 0.8773
Epoch 26/100
106/106 [==============================] - 0s 987us/step  - loss: 0.3346  - accurac
y: 0.8603 - val_loss: 0.3152 - val_accuracy: 0.8853
Epoch 27/100
106/106 [==============================] - 0s 970us/step  - loss: 0.3338  - accurac
y: 0.8641 - val_loss: 0.3153 - val_accuracy: 0.8827
Epoch 28/100
106/106 [==============================] - 0s 981us/step  - loss: 0.3328  - accurac
y: 0.8609 - val_loss: 0.3122 - val_accuracy: 0.8840
Epoch 29/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3326 - accuracy:
0.8622 - val_loss: 0.3204 - val_accuracy: 0.8693
Epoch 30/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3321 - accuracy:
0.8631 - val_loss: 0.3143 - val_accuracy: 0.8787
Epoch 31/100
106/106 [==============================] - 0s 977us/step - loss: 0.3316 - accurac
y: 0.8633 - val_loss: 0.3184 - val_accuracy: 0.8800
Epoch 32/100
106/106 [==============================] - 0s 995us/step - loss: 0.3309 - accurac
y: 0.8619 - val_loss: 0.3129 - val_accuracy: 0.8813
Epoch 33/100
106/106 [==============================] - 0s 993us/step - loss: 0.3314 - accurac
y: 0.8641 - val_loss: 0.3202 - val_accuracy: 0.8760
Epoch 34/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3299 - accuracy:
0.8659 - val_loss: 0.3144 - val_accuracy: 0.8840
Epoch 35/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3294 - accuracy:
0.8621 - val_loss: 0.3172 - val_accuracy: 0.8747
Epoch 36/100
106/106 [==============================] - 0s 974us/step - loss: 0.3305 - accurac
y: 0.8625 - val_loss: 0.3140 - val_accuracy: 0.8773
Epoch 37/100
106/106 [==============================] - 0s 960us/step - loss: 0.3286 - accurac
y: 0.8647 - val_loss: 0.3143 - val_accuracy: 0.8773
Epoch 38/100
106/106 [==============================] - 0s 958us/step - loss: 0.3289 - accurac
y: 0.8656 - val_loss: 0.3209 - val_accuracy: 0.8707
Epoch 39/100
106/106 [==============================] - 0s 958us/step - loss: 0.3285 - accurac
y: 0.8630 - val_loss: 0.3190 - val_accuracy: 0.8787
Epoch 40/100
106/106 [==============================] - 0s 996us/step - loss: 0.3288 - accurac
y: 0.8630 - val_loss: 0.3148 - val_accuracy: 0.8800
```

```
Epoch 41/100
106/106 [==============================] - 0s 960us/step - loss: 0.3266 - accurac
y: 0.8643 - val_loss: 0.3103 - val_accuracy: 0.8867
Epoch 42/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3273 - accuracy:
0.8653 - val_loss: 0.3151 - val_accuracy: 0.8787
Epoch 43/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3271 - accuracy:
0.8658 - val_loss: 0.3149 - val_accuracy: 0.8787
Epoch 44/100
106/106 [==============================] - 0s 984us/step - loss: 0.3259 - accurac
y: 0.8673 - val_loss: 0.3196 - val_accuracy: 0.8707
Epoch 45/100
106/106 [==============================] - 0s 962us/step - loss: 0.3259 - accurac
y: 0.8671 - val_loss: 0.3196 - val_accuracy: 0.8773
Epoch 46/100
106/106 [==============================] - 0s 960us/step - loss: 0.3250 - accurac
y: 0.8656 - val_loss: 0.3145 - val_accuracy: 0.8827
Epoch 47/100
106/106 [==============================] - 0s 976us/step - loss: 0.3246 - accurac
y: 0.8679 - val_loss: 0.3102 - val_accuracy: 0.8813
Epoch 48/100
106/106 [==============================] - 0s 992us/step - loss: 0.3245 - accurac
y: 0.8670 - val_loss: 0.3181 - val_accuracy: 0.8840
Epoch 49/100
106/106 [==============================] - 0s 995us/step - loss: 0.3246 - accurac
y: 0.8668 - val_loss: 0.3167 - val_accuracy: 0.8773
Epoch 50/100
106/106 [==============================] - 0s 958us/step - loss: 0.3236 - accurac
y: 0.8643 - val_loss: 0.3197 - val_accuracy: 0.8733
Epoch 51/100
106/106 [==============================] - 0s 1ms/step  - loss: 0.3230 - accuracy:
0.8668 - val_loss: 0.3130 - val_accuracy: 0.8787
Epoch 52/100
106/106 [==============================] - 0s 1ms/step  - loss: 0.3227 - accuracy:
0.8664 - val_loss: 0.3142 - val_accuracy: 0.8800
Epoch 53/100
106/106 [==============================] - 0s 1ms/step  - loss: 0.3225 - accuracy:
0.8665 - val_loss: 0.3105 - val_accuracy: 0.8840
Epoch 54/100
106/106 [==============================] - 0s 1ms/step  - loss: 0.3226 - accuracy:
0.8664 - val_loss: 0.3181 - val_accuracy: 0.8773
Epoch 55/100
106/106 [==============================] - 0s 976us/step - loss: 0.3219 - accurac
y: 0.8661 - val_loss: 0.3144 - val_accuracy: 0.8813
Epoch 56/100
106/106 [==============================] - 0s 964us/step - loss: 0.3220 - accurac
y: 0.8698 - val_loss: 0.3190 - val_accuracy: 0.8733
Epoch 57/100
```

```
106/106 [==============================] - 0s 1ms/step - loss: 0.3217 - accuracy:
0.8664 - val_loss: 0.3128 - val_accuracy: 0.8800
Epoch 58/100
106/106 [==============================] - 0s 967us/step - loss: 0.3208 - accurac
y: 0.8692 - val_loss: 0.3240 - val_accuracy: 0.8733
Epoch 59/100
106/106 [==============================] - 0s 972us/step - loss: 0.3199 - accurac
y: 0.8665 - val_loss: 0.3254 - val_accuracy: 0.8653
Epoch 60/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3197 - accuracy:
0.8698 - val_loss: 0.3177 - val_accuracy: 0.8800
Epoch 61/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3196 - accuracy:
0.8681 - val_loss: 0.3106 - val_accuracy: 0.8787
Epoch 62/100
106/106 [==============================] - 0s 983us/step - loss: 0.3197 - accurac
y: 0.8692 - val_loss: 0.3194 - val_accuracy: 0.8707
Epoch 63/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3196 - accuracy:
0.8670 - val_loss: 0.3156 - val_accuracy: 0.8853
Epoch 64/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3177 - accuracy:
0.8695 - val_loss: 0.3168 - val_accuracy: 0.8747
Epoch 65/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3189 - accuracy:
0.8686 - val_loss: 0.3176 - val_accuracy: 0.8747
Epoch 66/100
106/106 [==============================] - 0s 974us/step - loss: 0.3183 - accurac
y: 0.8662 - val_loss: 0.3155 - val_accuracy: 0.8827
Epoch 67/100
106/106 [==============================] - 0s 998us/step - loss: 0.3171 - accurac
y: 0.8699 - val_loss: 0.3154 - val_accuracy: 0.8827
Epoch 68/100
106/106 [==============================] - 0s 986us/step - loss: 0.3169 - accurac
y: 0.8683 - val_loss: 0.3215 - val_accuracy: 0.8733
Epoch 69/100
106/106 [==============================] - 0s 976us/step - loss: 0.3161 - accurac
y: 0.8649 - val_loss: 0.3193 - val_accuracy: 0.8707
Epoch 70/100
106/106 [==============================] - 0s 983us/step - loss: 0.3164 - accurac
y: 0.8720 - val_loss: 0.3187 - val_accuracy: 0.8813
Epoch 71/100
106/106 [==============================] - 0s 973us/step - loss: 0.3159 - accurac
y: 0.8704 - val_loss: 0.3193 - val_accuracy: 0.8773
Epoch 72/100
106/106 [==============================] - 0s 999us/step - loss: 0.3148 - accurac
y: 0.8710 - val_loss: 0.3163 - val_accuracy: 0.8813
Epoch 73/100
106/106 [==============================] - 0s 993us/step - loss: 0.3151 - accurac
y: 0.8707 - val_loss: 0.3193 - val_accuracy: 0.8733
Epoch 74/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3142 - accuracy:
0.8704 - val_loss: 0.3145 - val_accuracy: 0.8800
Epoch 75/100
106/106 [==============================] - 0s 995us/step - loss: 0.3150 - accurac
y: 0.8690 - val_loss: 0.3197 - val_accuracy: 0.8773
Epoch 76/100
106/106 [==============================] - 0s 975us/step - loss: 0.3132 - accurac
y: 0.8704 - val_loss: 0.3139 - val_accuracy: 0.8800
Epoch 77/100
```

```
106/106 [==============================] - 0s 1ms/step - loss: 0.3138 - accuracy:
0.8714 - val_loss: 0.3149 - val_accuracy: 0.8827
Epoch 78/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3126 - accuracy:
0.8735 - val_loss: 0.3173 - val_accuracy: 0.8733
Epoch 79/100
106/106 [==============================] - 0s 993us/step - loss: 0.3124 - accurac
y: 0.8720 - val_loss: 0.3214 - val_accuracy: 0.8747
Epoch 80/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3121 - accuracy:
0.8724 - val_loss: 0.3169 - val_accuracy: 0.8787
Epoch 81/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3120 - accuracy:
0.8705 - val_loss: 0.3157 - val_accuracy: 0.8760
Epoch 82/100
106/106 [==============================] - 0s 992us/step - loss: 0.3112 - accurac
y: 0.8730 - val_loss: 0.3220 - val_accuracy: 0.8693
Epoch 83/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3109 - accuracy:
0.8699 - val_loss: 0.3220 - val_accuracy: 0.8640
Epoch 84/100
106/106 [==============================] - 0s 994us/step - loss: 0.3100 - accurac
y: 0.8713 - val_loss: 0.3203 - val_accuracy: 0.8787
Epoch 85/100
106/106 [==============================] - 0s 975us/step - loss: 0.3100 - accurac
y: 0.8747 - val_loss: 0.3192 - val_accuracy: 0.8827
Epoch 86/100
106/106 [==============================] - 0s 957us/step - loss: 0.3107 - accurac
y: 0.8736 - val_loss: 0.3216 - val_accuracy: 0.8733
Epoch 87/100
106/106 [==============================] - 0s 953us/step - loss: 0.3091 - accurac
y: 0.8729 - val_loss: 0.3158 - val_accuracy: 0.8813
Epoch 88/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3088 - accuracy:
0.8730 - val_loss: 0.3256 - val_accuracy: 0.8733
Epoch 89/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3090 - accuracy:
0.8744 - val_loss: 0.3178 - val_accuracy: 0.8693
Epoch 90/100
106/106 [==============================] - 0s 974us/step - loss: 0.3079 - accurac
y: 0.8736 - val_loss: 0.3209 - val_accuracy: 0.8720
Epoch 91/100
106/106 [==============================] - 0s 993us/step - loss: 0.3072 - accurac
y: 0.8744 - val_loss: 0.3149 - val_accuracy: 0.8747
Epoch 92/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3077 - accuracy:
0.8705 - val_loss: 0.3243 - val_accuracy: 0.8667
Epoch 93/100
106/106 [==============================] - 0s 992us/step - loss: 0.3069 - accurac
y: 0.8720 - val_loss: 0.3204 - val_accuracy: 0.8680
Epoch 94/100
106/106 [==============================] - 0s 989us/step - loss: 0.3062 - accurac
y: 0.8747 - val_loss: 0.3175 - val_accuracy: 0.8760
Epoch 95/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3060 - accuracy:
0.8741 - val_loss: 0.3171 - val_accuracy: 0.8733
Epoch 96/100
106/106 [==============================] - 0s 997us/step - loss: 0.3052 - accurac
y: 0.8736 - val_loss: 0.3198 - val_accuracy: 0.8733
Epoch 97/100
```

```
106/106 [==============================] - 0s 970us/step - loss: 0.3042 - accurac
y: 0.8741 - val_loss: 0.3363 - val_accuracy: 0.8707
Epoch 98/100
106/106 [==============================] - 0s 982us/step - loss: 0.3056 - accurac
y: 0.8750 - val_loss: 0.3190 - val_accuracy: 0.8747
Epoch 99/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3041 - accuracy:
0.8724 - val_loss: 0.3175 - val_accuracy: 0.8827
Epoch 100/100
106/106 [==============================] - 0s 1ms/step - loss: 0.3043 - accuracy:
0.8724 - val_loss: 0.3212 - val_accuracy: 0.8760
```

Out[20]: <keras.callbacks.History at 0x7f488811da00>

In [21]:
```
pred = model.predict(x_test)
```

```
79/79 [==============================] - 0s 567us/step
```

In [22]:
```
y_pred = []
for val in pred:
    if val > 0.5:
        y_pred.append(1)
    else:
        y_pred.append(0)
```

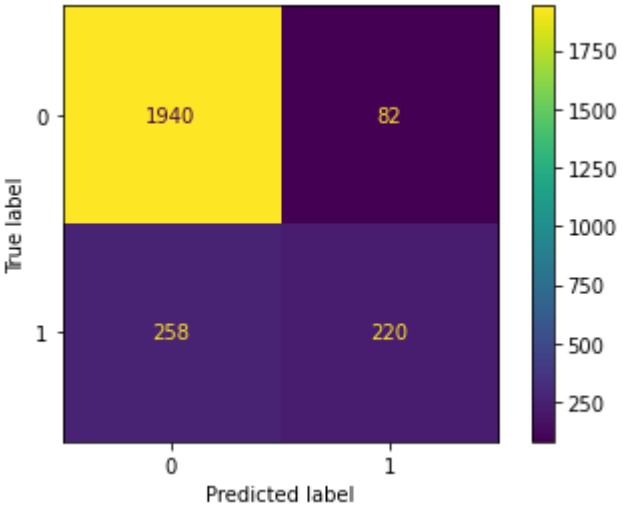
In [23]:
```
from sklearn.metrics import accuracy_score,confusion_matrix,ConfusionMatrixDispl
```

In [24]:
```
accuracy_score(y_test,y_pred)
```

Out[24]: 0.864

In [25]:
```
cm = confusion_matrix(y_test,y_pred)
display = ConfusionMatrixDisplay(cm)
display.plot()
```

Out[25]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7f487c0f38e0>



In [26]:
```
from sklearn.neural_network import MLPClassifier
```

In [ ]:
```
nn_classifier = MLPClassifier(hidden_layer_sizes=(100),activation='logistic',max
nn_classifier.fit(x_train,y_train)
```

In [50]:
```
y_pred2 = nn_classifier.predict(x_test)
```

In [51]:
```
accuracy_score(y_pred=y_pred2,y_true=y_test)
```

Out[51]: 0.862

In [52]:
```
nn_classifier.score(x_test,y_test)
```

Out[52]: 0.862

In [ ]:
```

```

Classify emails as spam or not spam using K-Nearest Neighbors and Support Vector Machine, and compare their performance.

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [3]:
```
df = pd.read_csv('emails.csv')
df.head()
```

Out[3]:

	Email No.	the	to	ect	and	for	of	a	you	hou	...	connevey	jay	valued	lay	i
0	Email 1	0	0	1	0	0	0	2	0	0	...	0	0	0	0	
1	Email 2	8	13	24	6	6	2	102	1	27	...	0	0	0	0	
2	Email 3	0	0	1	0	0	0	8	0	0	...	0	0	0	0	
3	Email 4	0	5	22	0	5	1	51	2	10	...	0	0	0	0	
4	Email 5	7	6	17	1	5	2	57	0	9	...	0	0	0	0	

5 rows × 3002 columns

In [5]:
```
df.isnull().sum()
```

Out[5]:

In [8]:
```
from sklearn.metrics import ConfusionMatrixDisplay,confusion_matrix,accuracy_sco
def report(classifier):
    y_pred = classifier.predict(x_test)
    cm = confusion_matrix(y_test,y_pred)
    display = ConfusionMatrixDisplay(cm,display_labels=classifier.classes_)
    display.plot()
```

In [6]:

In [7]:

```
Email
No.

0
the

0
to

0
ect

0
and

0
              .
              .
military       0
allowing       0
ff             0
dry            0
Prediction     0
Length: 3002, dtype: int64
```

```
df.dropna(how='any',inplace=True)
```

```
x = df.iloc[:,1:-1].values
y = df.iloc[:,-1].values
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test  =  train_test_split(x,y,test_size=0.25,random_state
```

```
    print(f"Accuracy:  {accuracy_score(y_test,y_pred)}")
    print(f"Precision Score: {precision_score(y_test,y_pred)}")
    print(f"Recall Score: {recall_score(y_test,y_pred)}")
    plot_precision_recall_curve(classifier,x_test,y_test)
    plot_roc_curve(classifier,x_test,y_test)
```

K-Nearest Neighbours Classifier

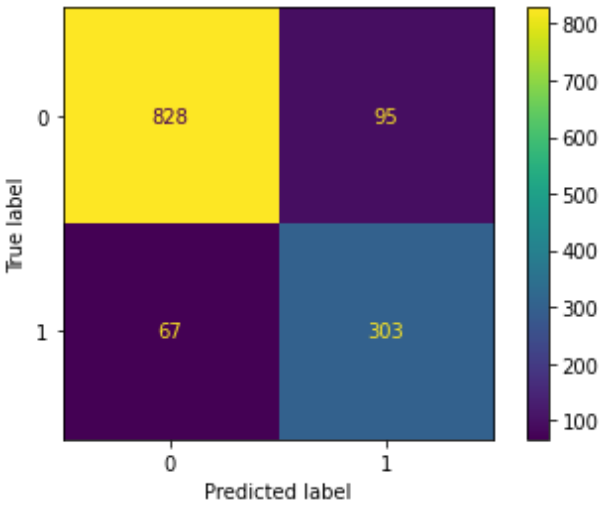
In [69]:
```
from sklearn.neighbors import KNeighborsClassifier
```

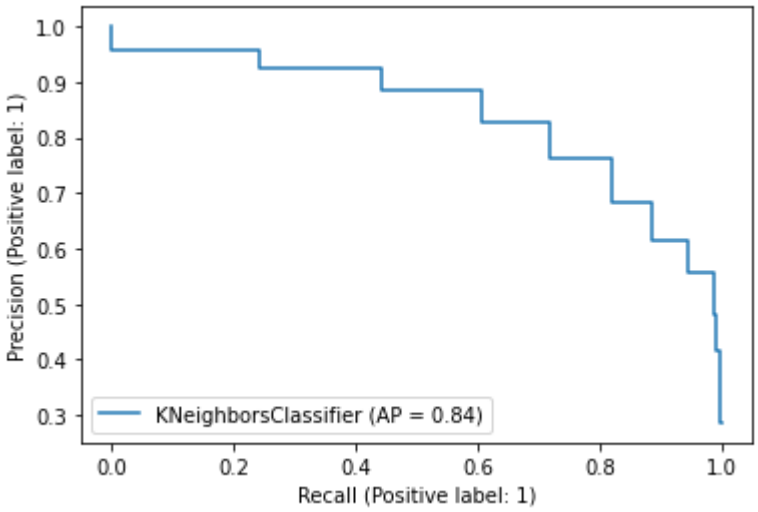
In [70]:
```
kNN = KNeighborsClassifier(n_neighbors=10)
kNN.fit(x_train,y_train)
```

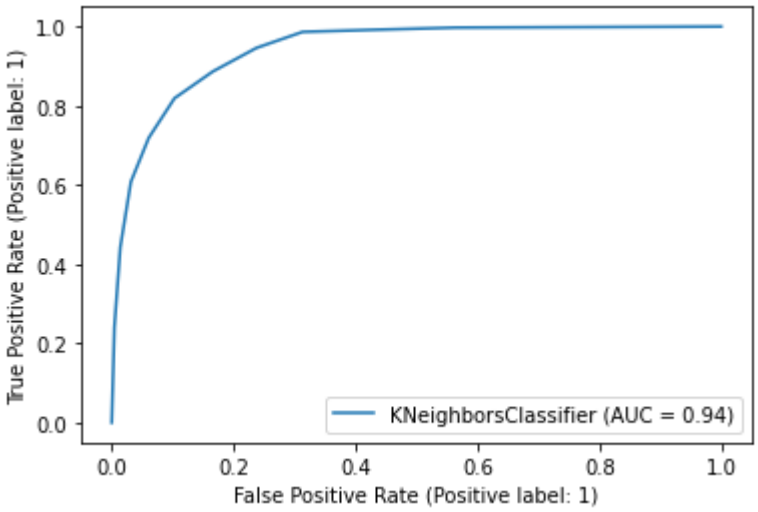
Out[70]:
```
KNeighborsClassifier(n_neighbors=10)
```

In [71]:
```
report(kNN)
```

```
Accuracy:  0.8747099767981439
Precision Score:  0.7613065326633166
Recall Score:  0.8189189189189189
/home/pratik/.local/lib/python3.8/site-packages/sklearn/utils/deprecation.py:87:
FutureWarning: Function plot_precision_recall_curve is deprecated; Function `plot
_precision_recall_curve` is deprecated in 1.0 and will be removed in 1.2. Use one of
the class methods: PrecisionRecallDisplay.from_predictions or PrecisionRecallD
isplay.from_estimator.
  warnings.warn(msg, category=FutureWarning)
/home/pratik/.local/lib/python3.8/site-packages/sklearn/utils/deprecation.py:87:
FutureWarning: Function plot_roc_curve is deprecated; Function :func:`plot_roc_cu
rve` is deprecated in 1.0 and will be removed in 1.2. Use one of the class method
s: :meth:`sklearn.metrics.RocCurveDisplay.from_predictions` or :meth:`sklearn.met
rics.RocCurveDisplay.from_estimator`.
  warnings.warn(msg, category=FutureWarning)
```







In [74]:
```
from sklearn.svm import SVC
svm = SVC(gamma='auto',random_state=10)
svm.fit(x_train,y_train)
```

Out[74]:

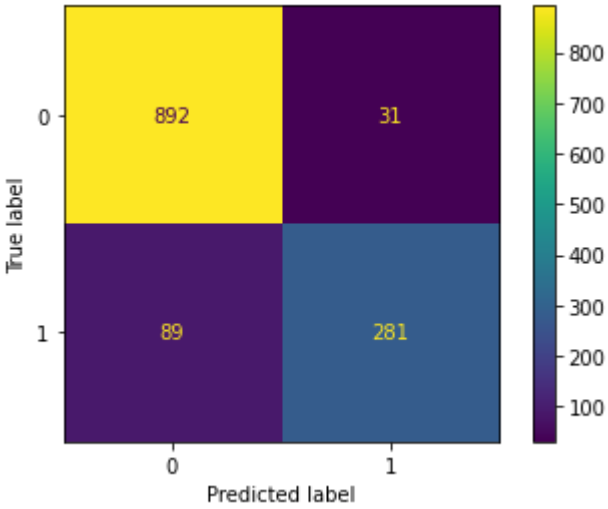
SVC

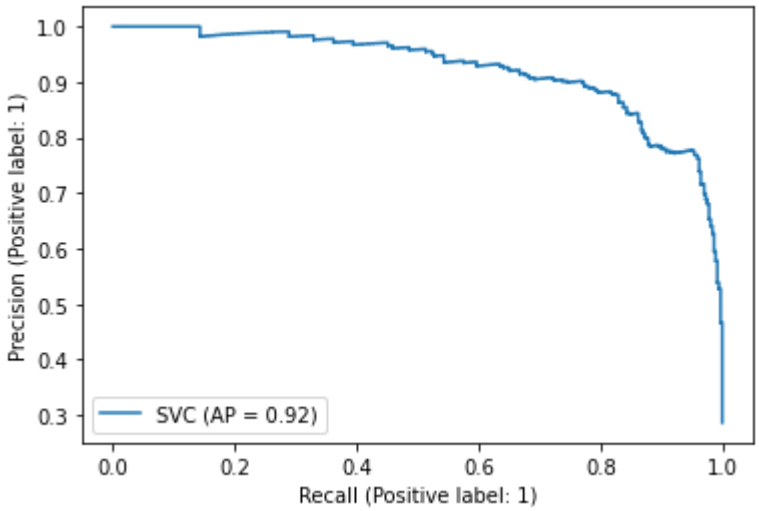
```
SVC(gamma='auto', random_state=10)
```

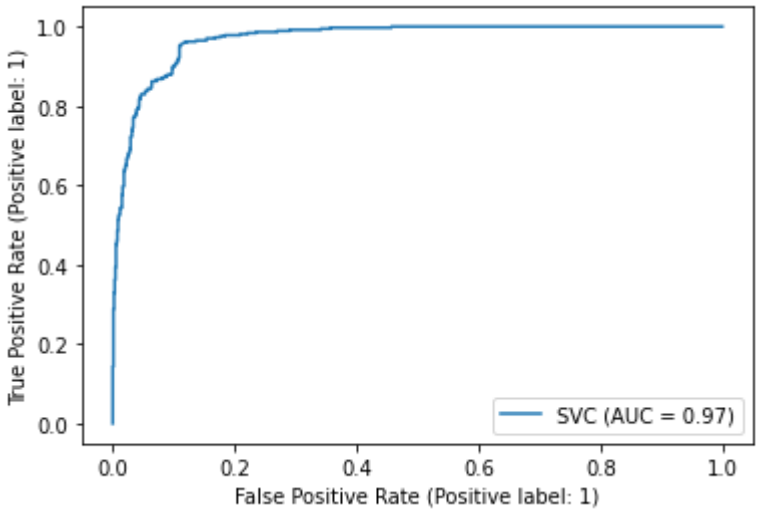
In [75]:
```
report(svm)
```

```
Accuracy:  0.9071925754060325
Precision Score:  0.9006410256410257
Recall Score:  0.7594594594594595
```

```
/home/pratik/.local/lib/python3.8/site-packages/sklearn/utils/deprecation.py:87:
FutureWarning: Function plot_precision_recall_curve is deprecated; Function `plot
_precision_recall_curve` is deprecated in 1.0 and will be removed in 1.2. Use one of
the class methods: PrecisionRecallDisplay.from_predictions or PrecisionRecallD
isplay.from_estimator.
  warnings.warn(msg, category=FutureWarning)
/home/pratik/.local/lib/python3.8/site-packages/sklearn/utils/deprecation.py:87:
FutureWarning: Function plot_roc_curve is deprecated; Function :func:`plot_roc_cu
rve` is deprecated in 1.0 and will be removed in 1.2. Use one of the class method
s: :meth:`sklearn.metrics.RocCurveDisplay.from_predictions` or :meth:`sklearn.met
rics.RocCurveDisplay.from_estimator`.
  warnings.warn(msg, category=FutureWarning)
```


892
31
89
281
0
1
True label
0
1
Predicted label
800
700
600
500
400
300
200
100


Precision (Positive label: 1)
Recall (Positive label: 1)
SVC (AP = 0.92)
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.0
0.2
0.4
0.6
0.8
1.0


True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
SVC (AUC = 0.97)
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

In [ ]:

Write and deploy a smart contract on a test network for a customer bank account supporting deposit, withdrawal, and balance display.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Bank {
    mapping(address => uint256) public balanceOf;

    // Deposit Ether into the bank
    function deposit() public payable {
        require(msg.value > 0, "Deposit should be greater than zero.");

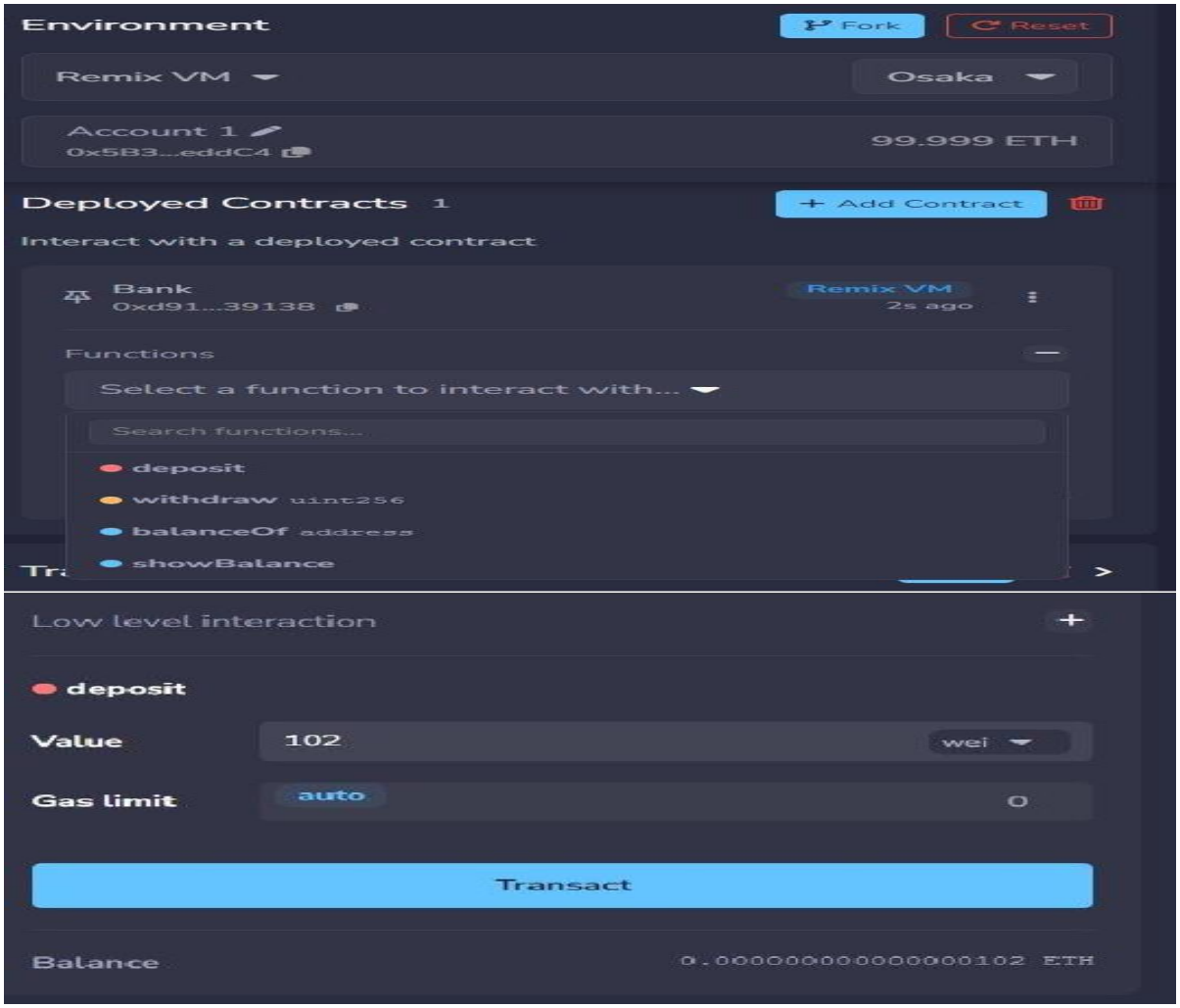
        balanceOf[msg.sender] += msg.value;
    }
    // Withdraw Ether from the bank
function withdraw(uint256 amount) public {
    require(amount > 0, "Amount must be greater than zero.");
    require(
        balanceOf[msg.sender] >= amount,
        "Insufficient balance."
    );

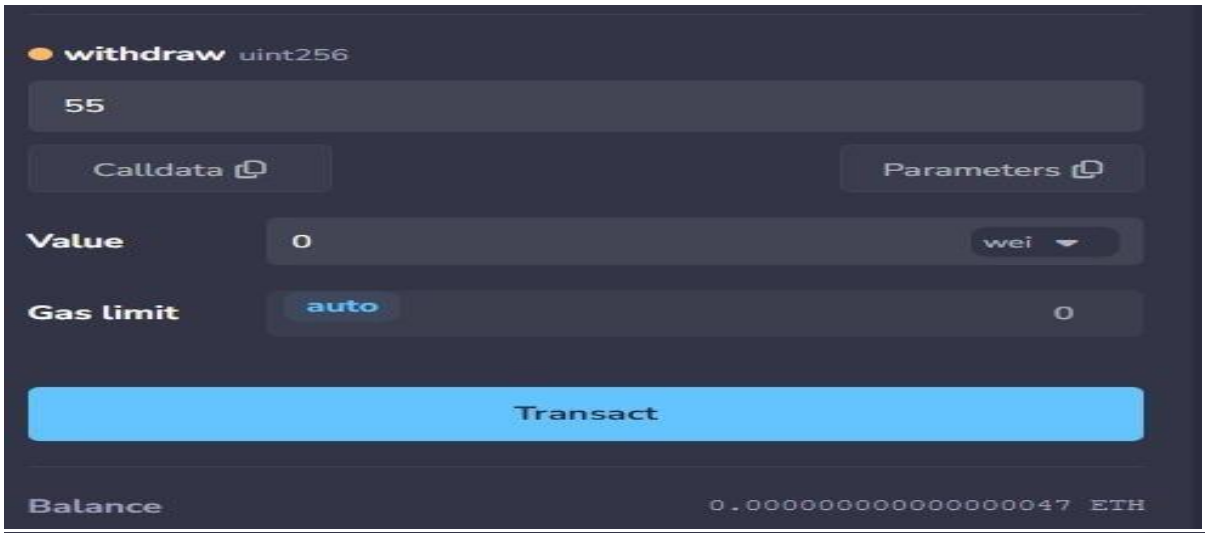
    balanceOf[msg.sender] -= amount;

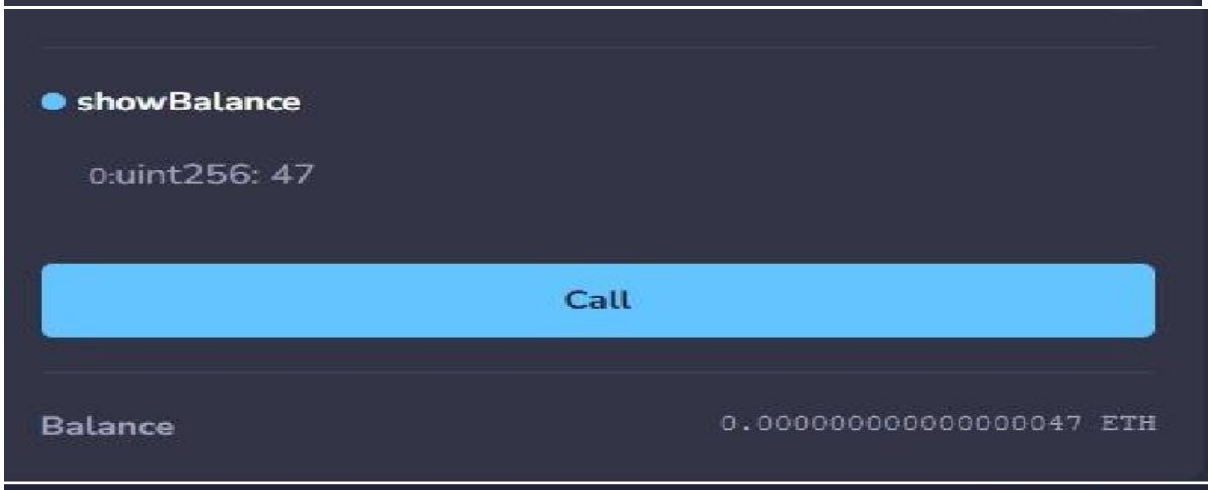
    (bool success, ) =
        payable(msg.sender).call{value: amount}("");

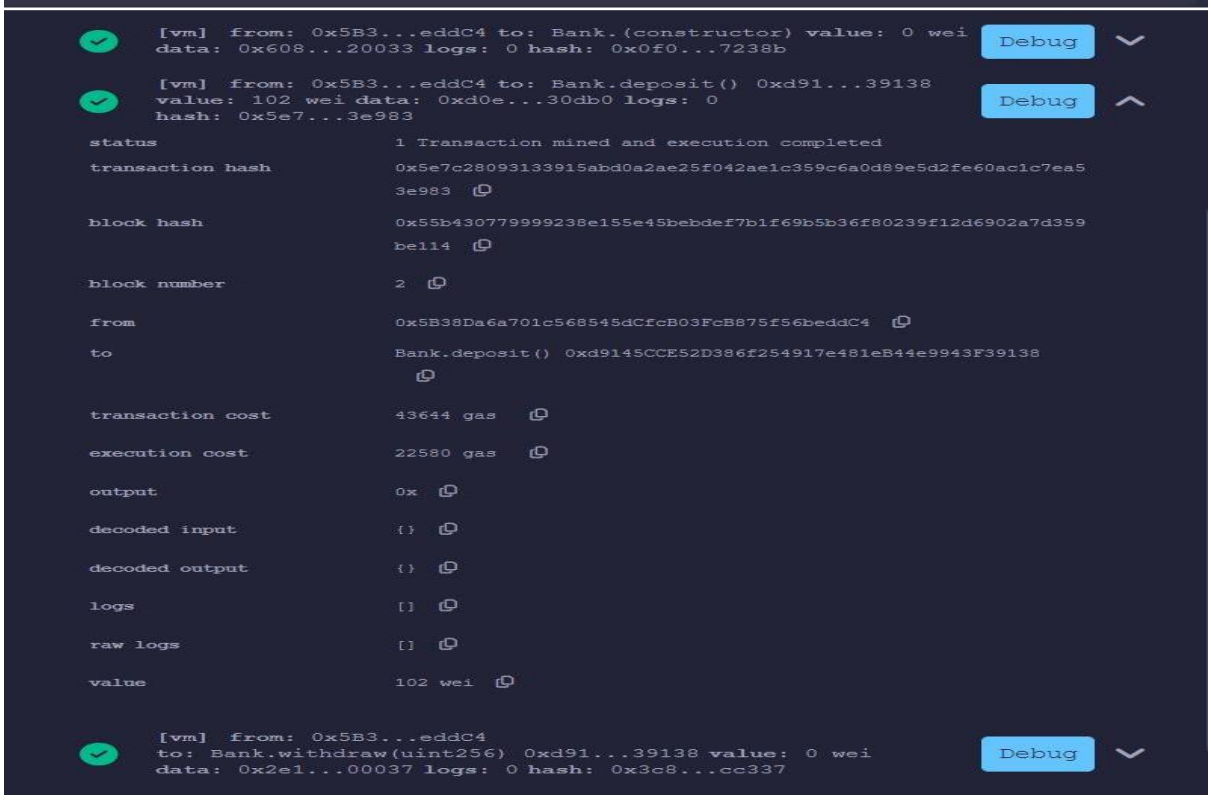
    require(success, "Ether transfer failed.");
}

    // Show the caller's balance
    function showBalance() public view returns (uint256) {
        return balanceOf[msg.sender];
    }
}
```


Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deployed Contracts 1
+ Add Contract
Interact with a deployed contract
Bank
0xd91...39138
Remix VM
2s ago
Functions
Select a function to interact with...
Search functions...
deposit
withdraw uint256
balanceOf address
showBalance
Low level interaction
deposit
Value
102
wei
Gas limit
auto
0
Transact
Balance
0.000000000000000102 ETH


withdraw uint256
55
Calldata
Parameters
Value
0
wei
Gas limit
auto
0
Transact
Balance
0.000000000000000047 ETH


showBalance
0:uint256: 47
Call
Balance
0.000000000000000047 ETH


[vm] from: 0x5B3...eddC4 to: Bank.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x0f0...7238b
Debug
[vm] from: 0x5B3...eddC4 to: Bank.deposit() 0xd91...39138 value: 102 wei data: 0xd0e...30db0 logs: 0 hash: 0x5e7...3e983
Debug
status 1 Transaction mined and execution completed
transaction hash 0x5e7c28093133915abd0a2ae25f042ae1c359c6a0d89e5d2fe60ac1c7ea53e983
block hash 0x55b430779999238e155e45bebdef7b1f69b5b36f80239f12d6902a7d359be114
block number 2
from 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to Bank.deposit() 0xd9145CCE52D386f254917e481eB44e9943F39138
transaction cost 43644 gas
execution cost 22580 gas
output 0x
decoded input {}
decoded output {}
logs []
raw logs []
value 102 wei
[vm] from: 0x5B3...eddC4 to: Bank.withdraw(uint256) 0xd91...39138 value: 0 wei data: 0x2e1...00037 logs: 0 hash: 0x3c8...cc337
Debug

Write a Solidity program to create student data using structures, arrays, and fallback; deploy it on Ethereum and observe transaction fee and gas values.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract StudentRegistry {
    struct Student {
        uint256 id;
        string name;
        uint8 age;
        string dept;
    }

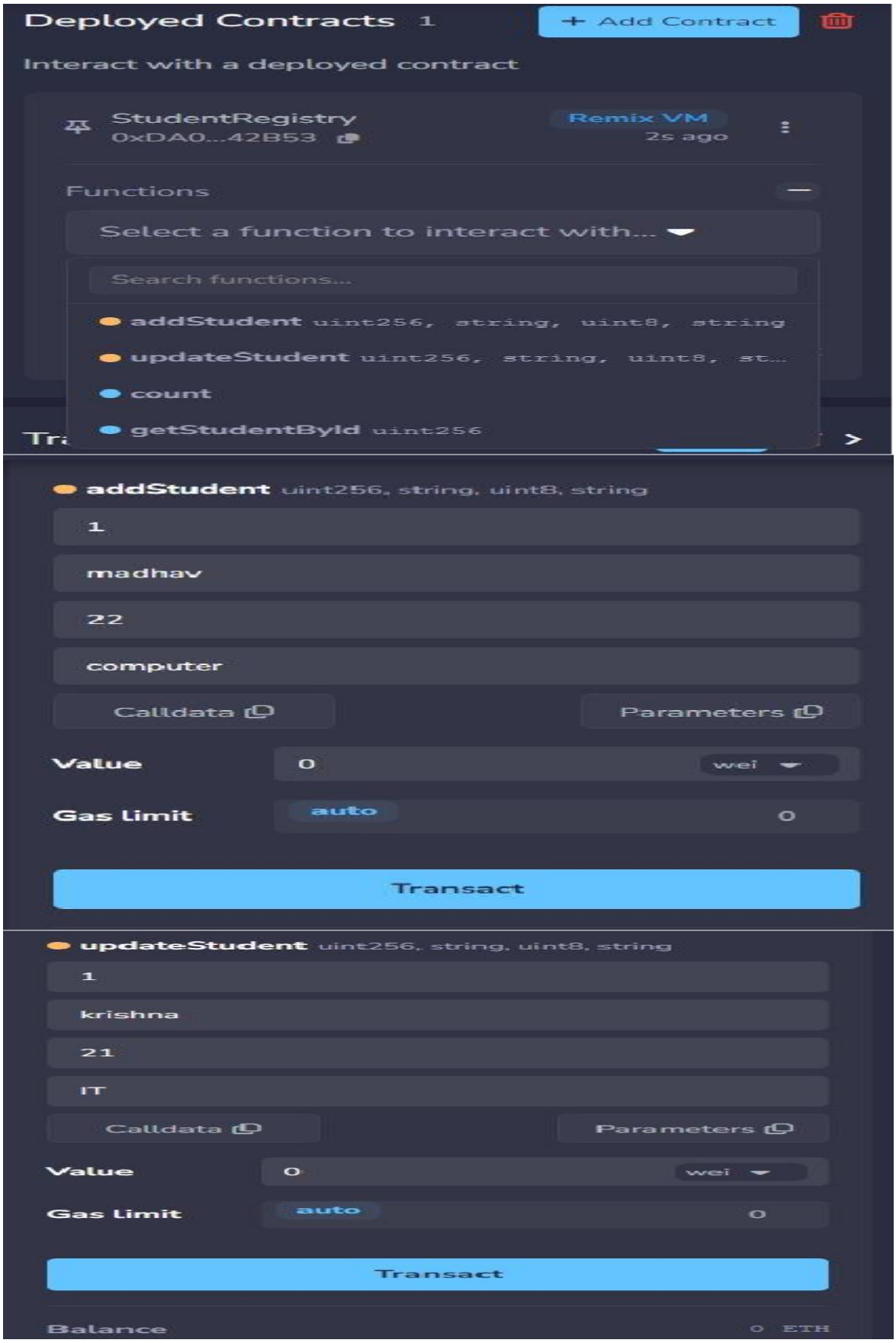
    Student[] private students;
    mapping(uint256 => uint256) private idToIndex; // id -> index+1
    event StudentAdded(uint256 id, string name);
    event StudentUpdated(uint256 id, string name);
    event EtherReceived(address indexed from, uint256 amount);
    event FallbackCalled(address indexed from, uint256 value, bytes data);

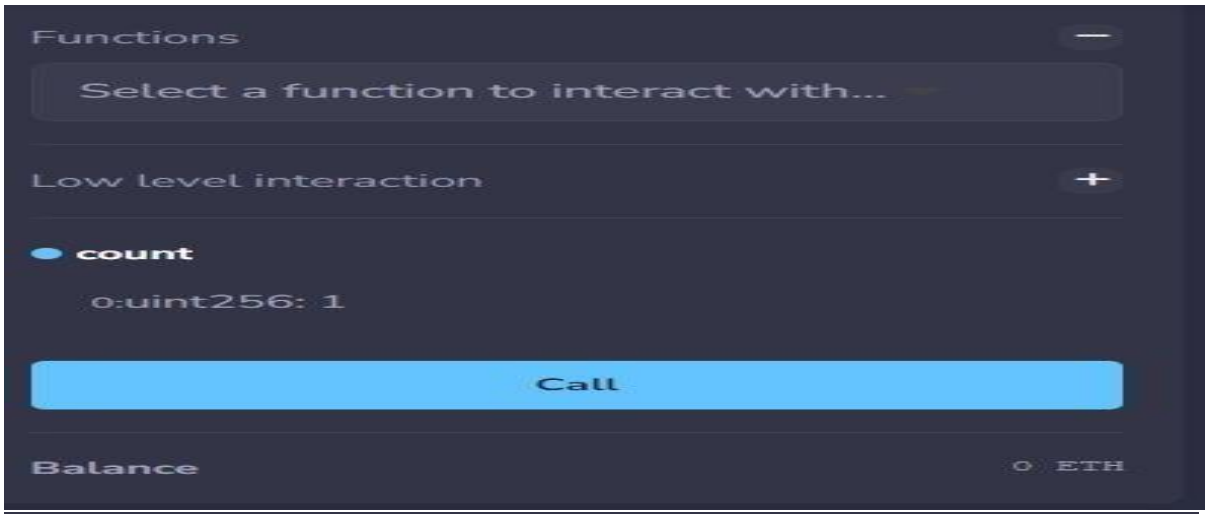
    receive() external payable {
        emit EtherReceived(msg.sender, msg.value);
    }

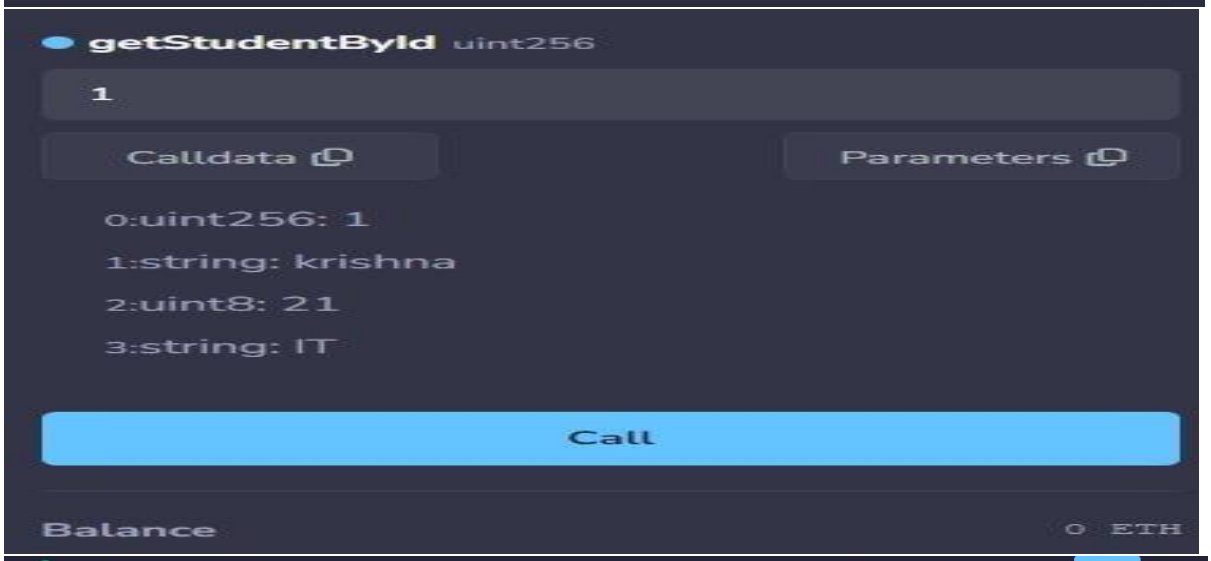
    fallback() external payable {
        emit FallbackCalled(msg.sender, msg.value, msg.data);
    }

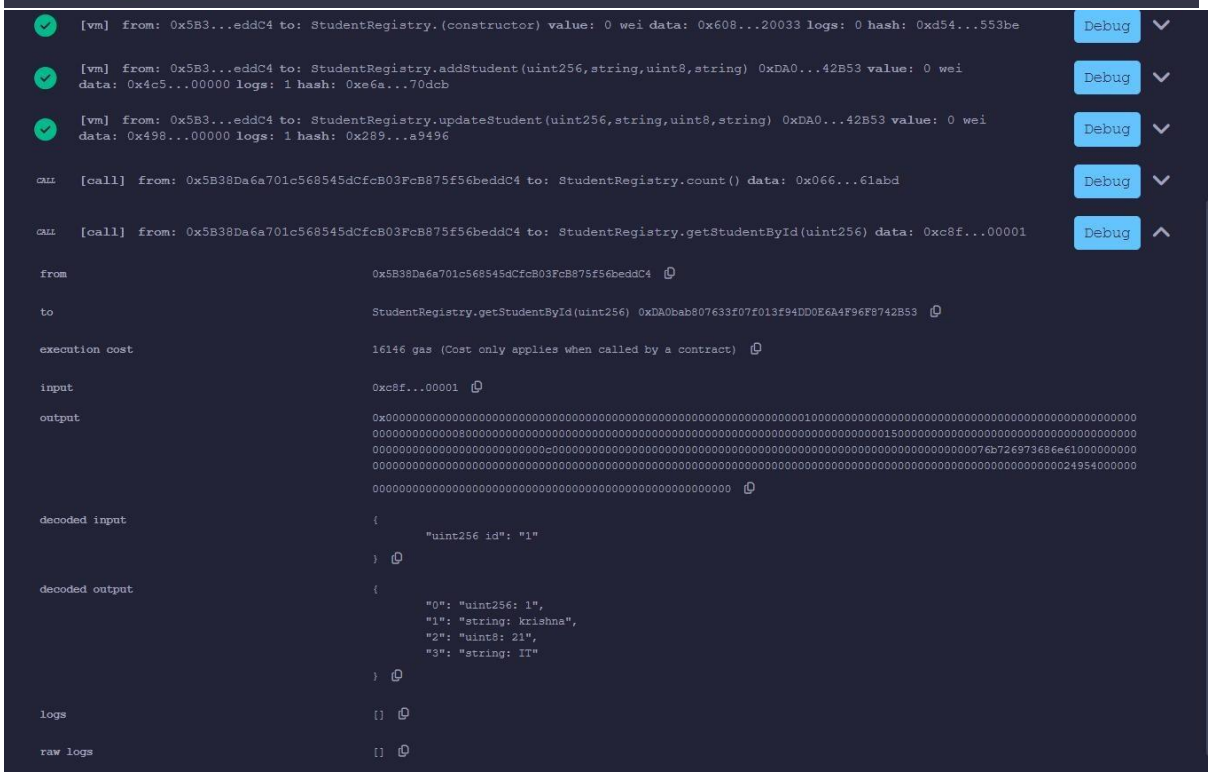
    function addStudent(
        uint256 id,
        string calldata name,
        uint8 age,
        string calldata dept
    ) external {
        require(idToIndex[id] == 0, "ID exists");
        students.push(Student(id, name, age, dept));
        idToIndex[id] = students.length; // store index+1
```

```
36            emit StudentAdded(id, name);
37        }
38
39        function updateStudent(
40            uint256 id,
41            string calldata name,
42            uint8 age,
43            string calldata dept
44        ) external {
45            uint256 idx = idToIndex[id];
46            require(idx != 0, "Not found");
47            Student storage s = students[idx - 1];
48            s.name = name;
49            s.age = age;
50            s.dept = dept;
51            emit StudentUpdated(id, name);
52        }
53
54        function getStudentById(uint256 id)
55            external
56            view
57            returns (uint256, string memory, uint8, string memory)
58        {
59            uint256 idx = idToIndex[id];
60            require(idx != 0, "Not found");
61            Student memory s = students[idx - 1];
62            return (s.id, s.name, s.age, s.dept);
63        }
64
65        function count() external view returns (uint256) {
66            return students.length;
67        }
68    }
```


Deployed Contracts 1
+ Add Contract
Interact with a deployed contract
StudentRegistry
0xDA0...42B53
Remix VM
2s ago
Functions
Select a function to interact with...
Search functions...
addStudent uint256, string, uint8, string
updateStudent uint256, string, uint8, st...
count
getStudentById uint256
addStudent uint256, string, uint8, string
1
madhav
22
computer
Calldata
Parameters
Value
0
wei
Gas limit
auto
0
Transact
updateStudent uint256, string, uint8, string
1
krishna
21
IT
Calldata
Parameters
Value
0
wei
Gas limit
auto
0
Transact
Balance
0 ETH


Functions
Select a function to interact with...
Low level interaction
count
0:uint256: 1
Call
Balance
0 ETH


getStudentById uint256
1
Calldata
Parameters
0:uint256: 1
1:string: krishna
2:uint8: 21
3:string: IT
Call
Balance
0 ETH


[vm] from: 0x5B3...eddC4 to: StudentRegistry.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xd54...553be
Debug
[vm] from: 0x5B3...eddC4 to: StudentRegistry.addStudent(uint256,string,uint8,string) 0xDA0...42B53 value: 0 wei
data: 0x4c5...00000 logs: 1 hash: 0xe6a...70dcb
Debug
[vm] from: 0x5B3...eddC4 to: StudentRegistry.updateStudent(uint256,string,uint8,string) 0xDA0...42B53 value: 0 wei
data: 0x498...00000 logs: 1 hash: 0x289...a9496
Debug
call [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: StudentRegistry.count() data: 0x066...61abd
Debug
call [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: StudentRegistry.getStudentById(uint256) data: 0xc8f...00001
Debug
from
0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to
StudentRegistry.getStudentById(uint256) 0xDA0bab807633f07f013f94DD0E6A4F96F8742B53
execution cost
16146 gas (Cost only applies when called by a contract)
input
0xc8f...00001
output
0x0000000000000000000000000000000000000000000000000000000000000010000000000000000000000000000000000000000000000000
0000000000000800000000000000000000000000000000000000000000000000000000000000150000000000000000000000000000000000000
0000000000000000000000000c000000000000000000000000000000000000000000000000000000000000076b726973686e6100000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000024954000000
0000000000000000000000000000000000000000000000000000000000
decoded input
{
"uint256 id": "1"
}
decoded output
{
"0": "uint256: 1",
"1": "string: krishna",
"2": "uint8: 21",
"3": "string: IT"
}
logs
[]
raw logs
[]